

Research Placement Final Report

Imitation Learning for Robotic Tracking of Human Wrist Trajectories

João Augusto Lopes

Supervisor: Maria Koskinopoulou

December 2024

1 Abstract

This project explores robotic imitation learning for tracking human wrist trajectories. It focuses on real-time control of the Franka Emika Panda robotic arm using data from an Intel RealSense RGB-D camera. The initial aim was to implement a system based on Learning from Demonstration (LfD), enabling the robot to autonomously replicate complex human motions. Due to time and resource constraints, the objective shifted toward developing a simplified real-time motion tracking solution. The system identifies wrist positions using a color-based tracking algorithm and translates this data into robot end-effector movements. ROS, Gazebo, and MoveIt were used for implementation and testing. The results demonstrated smooth and accurate robot motion, with 82% motion planning success. Challenges included depth sensor inaccuracies, calibration difficulties, and simulation setup issues. The project provides a foundation for integrating advanced techniques like AI-based hand tracking in future iterations.

Contents

1	Abstract	2
2	Introduction	5
3	Aims and Objectives	6
3.1	Initial Objective	6
3.2	Final/New Objective	6
4	Project Planning	8
5	Background Theory	10
6	Design Tools	11
6.1	Hardware	11
6.1.1	Intel RealSense D435 RGB-D Camera	11
6.1.2	Franka Emika Panda Robotic Arm	11
6.2	Software	13
6.2.1	Robot Operating System	13
6.2.2	MoveIt	13
6.2.3	Gazebo	13
6.2.4	RViz	14
7	Implementation	15
7.1	System Design	15
7.2	Wrist Tracking	16
7.3	Panda Movement Calculation	19
7.4	Motion Planner	21
8	Results	23
8.1	Positional Analysis	23
8.2	Motion Planner Success	25
9	Challenges	27
10	Future Work	29
11	Conclusion	31

List of Figures

1	Project Initial Gantt Chart	9
2	Intel RealSense D435 RGB-D Camera	11
3	Franka Emika Panda	12
4	System Design	16
5	Tracking Visualisation on Rviz	17
6	Example of Tracked Points	18
7	3D points over time	24
8	Successful vs Failed Positions	25
9	AI Models	30

2 Introduction

Robotics research increasingly focuses on enabling robots to imitate human actions. Learning from Demonstration (LfD) is a promising approach that teaches robots to perform tasks based on human demonstrations. The initial goal of this project was to design an LfD-based system for robotic tracking of human wrist trajectories. This approach, while robust, required extensive data collection, model training, and complex integration. Given the constraints of time and resources, the project adopted a more streamlined goal. The new objective aimed to develop a real-time system for the Franka Emika Panda robotic arm to mimic human wrist movements. This was achieved using a straightforward motion tracking method, bypassing the need for advanced machine learning models. The system utilized an Intel RealSense RGB-D camera for wrist tracking and ROS tools for robotic control. Gazebo provided a simulated environment for initial testing, while MoveIt was used for motion planning and execution. Despite challenges, the project achieved smooth and accurate robotic movements, offering a practical framework for real-time human-robot interaction. This report outlines the design, implementation, results, challenges, and future directions of this work.

3 Aims and Objectives

3.1 Initial Objective

The initial goal for this placement was to develop a comprehensive system that employs the concept of "Learning from Demonstration" (LfD) to enable a robotic arm to replicate human actions such as moving and grasping objects. The underlying idea of LfD is to use machine learning algorithms to train robots based on data derived from human demonstrations. The system would interpret these demonstrations, learning how to perform similar tasks autonomously. This method, while effective, can be quite complex and time-consuming. Collecting the necessary training data, refining the model, and applying the learned information to real-world scenarios all contribute to an extended development period. The process of gathering and processing training samples is especially lengthy, as it requires a significant amount of both manual labour and computational resources.

Given these challenges and the need for efficiency, it became apparent that meeting the original objective within the constraints of the project timeline would be difficult. This realization led to the decision to reassess and simplify the overall goal, making it more achievable while still allowing room for meaningful progress.

3.2 Final/New Objective

The new objective focuses on a simpler approach to achieve a similar outcome. Instead of relying on the complex and time-intensive machine learning method, the decision was made to pivot toward a more practical and faster solution. The revised approach concentrates on developing a system in which a robotic arm mimics the movements of a human arm in real-time. This solution involves tracking and replicating basic human actions using more straightforward techniques, which can be implemented more quickly. By

doing so, the project timeline becomes more realistic, allowing for tangible results in a shorter period.

This new approach, although simpler, still maintains the core objective of enabling human-robot interaction and movement replication. Furthermore, it provides the flexibility to integrate Learning from Demonstration techniques into the system at a later stage, should time and resources permit. The revision ensures that the project stays on track while offering the potential to revisit and expand on the original concept.

4 Project Planning

Figure 1 shows the planned project timeline which spans from August 26, 2024, to the week of December 16, 2024, covering a total of 17 weeks. The project is structured into several key phases, each with specific tasks and milestones aimed at achieving the overall project objectives. The planning process ensures a systematic approach to managing time and resources.

The project begins with the Robotic Aspirator phase, which includes mechanical production and the creation of a detailed mechanical description report. Following this, Stage 1 focuses on wrist tracking, involving the setup of the Robot Operating System (ROS) and the research and implementation of tracking algorithms.

Stage 2 is dedicated to the simulation, in Gazebo, where the setup of the simulation environment and development of the simulation code is carried out. This phase is merely focused on simulating a Panda robot and assuring it can move its end-effector to a desired position. At this point, it doesn't integrate any of the wrist tracking outputs.

During Week 8, the submission of the interim report and presentation are required to be handed. Therefore their preparation and elaboration is required. This stage helps to summarize progress and make necessary adjustments.

Stage 3 involves integrating Gazebo simulation with wrist tracking and testing it on the configured Gazebo environment. In Stage 4, the project adapts the configuration to work with the physical Panda robot. This is followed by extensive testing to validate this integration.

The final phase of the project includes the production of a comprehensive final report and preparation for the final presentation, which showcases the project's outcomes and achievements. The timeline is meticulously planned, with each phase and task allocated specific weeks within the 17-week period, ensuring that all tasks are completed in a timely manner with clear milestones to track progress.

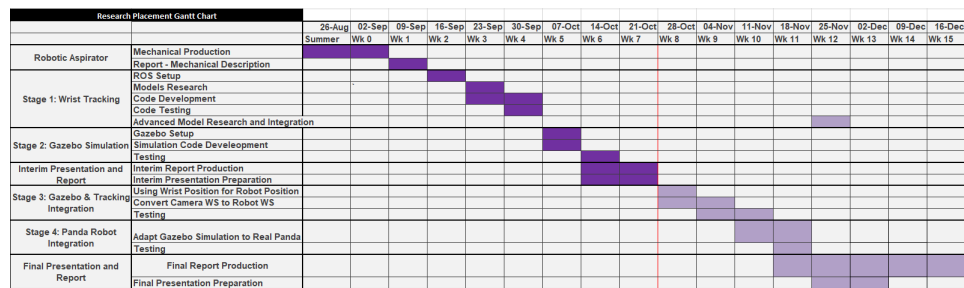


Figure 1: Project Initial Gantt Chart

5 Background Theory

6 Design Tools

6.1 Hardware

6.1.1 Intel RealSense D435 RGB-D Camera

To produce intricate 3D representations, the Intel RealSense D435 RGB-D Camera combines RGB imaging and depth sensing. It takes advantage of stereo vision technology, in which two sensors (visible in 2 take pictures from slightly different perspectives. By comparing these images, the camera determines the distance to each point in the image, producing a depth map. Furthermore, an infrared projector improves accuracy, especially in low light, by projecting a pattern onto the area. This RGB and depth data combination is perfect for a variety of applications since it enables accurate 3D modelling and object tracking [1].



Figure 2: Intel RealSense D435 RGB-D Camera

6.1.2 Franka Emika Panda Robotic Arm

The Franka Emika Panda is a state-of-the-art collaborative robot (cobot) built for accuracy and adaptability. With its 850 mm reach and 3 kg weight capacity, this 7-axis robotic arm may be used for a number of operations, like dispensing, welding, and remote TCP. The Panda is well known for its remarkable accuracy in tasks due to its excellent repeatability of 0.1 mm [2].

Developed by Franka Emika, the Panda is equipped with sophisticated sensors and control algorithms that give it exceptional agility and vision. Both experts and beginners can use it thanks to its intuitive programming and user-friendly interface, which speeds up deployment and reduces downtime. The robot is a flexible instrument that may be used in a variety of industrial and research settings. This is due to its design, which focuses on simplicity of integration with other systems. The Panda is a popular option for automation activities in a variety of industries due to its collaborative nature and advanced capabilities [3].



Figure 3: Franka Emika Panda

6.2 Software

6.2.1 Robot Operating System

Robot Operating System (ROS) 1 is an open-source framework used to develop robot software. It provides libraries and other tools such as drivers, cutting-edge algorithms, and developer tools to assist in the creation of robot applications. ROS facilitates component integration, allowing for smooth co-operation and communication between different parts of robotic systems [4]. Melodic Morenia was the ROS distribution used in this project. It was set up on the Ubuntu 18.04 release, which is the primary target for this particular ROS distribution.

6.2.2 MoveIt

MoveIt is an open-source software system for robotic motion. It provides a wide range of libraries and tools to support programmers to develop intricate manipulation applications. MoveIt simplifies the process of prototyping designs, benchmarking algorithms, and integrating with various robotic systems. Kinematic modelling, trajectory generation, collision checking, and motion planning and execution are among its capabilities [5]. MoveIt's extensible and flexible design makes it easy for different robotic system components to collaborate and communicate with one another. Its versatility makes it a valuable instrument for robotics research and business projects.

6.2.3 Gazebo

Developed by Open Robotics, Gazebo is an open-source 2D/3D robotics simulator. It offers a comprehensive collection of tools, libraries, and cloud services for modelling robots in elaborate and realistic settings [6]. With the help of high-fidelity physics and sensor models, rendering capabilities, and a variety of robot models, Gazebo makes it possible to accurately simulate intricate indoor and outdoor scenarios [7]. Gazebo is a useful tool for robotics

researchers and developers due to its robust and expandable architecture, which enables quick iterations on design concepts, control strategies, and algorithm testing.

6.2.4 RViz

RViz (ROS Visualization) is a visualization tool in ROS used for displaying robot models, sensor data, and environments. It provides a 3D interface to view robot states like joint positions and motion paths. Users can observe data such as laser scans, point clouds, and occupancy grids. The interface is customisable, allowing developers to add or adjust displays based on their needs. This flexibility makes it a valuable tool for debugging and understanding robot behaviour. RViz integrates well with ROS, helping developers verify robot functionality in both simulations and real-world settings. It simplifies testing and improves development efficiency [8].

Gazebo and RViz can be easily confused due their visual similarities, but they serve distinct purposes. Gazebo simulates a robot in a virtual environment, generating synthetic sensor data and simulating physics. RViz, on the other hand, visualizes the state of a robot, whether physical or simulated, by displaying its sensor data. "In other words, RViz shows you what the robot thinks is happening, while Gazebo (or looking at your physical robot) shows you what is really happening." [8]

7 Implementation

This section describes the process of launching the different components involved in tracking and controlling the Franka Emika Panda robot in a simulated environment. The implementation begins by launching the Gazebo and RViz environments, allowing real-time visualization of both the robot's movements and the RGB and masked images. Next, the motion planner node is initiated, enabling trajectory planning for the robot. The wrist tracking node follows, publishing the RGB and masked images required for tracking. Finally, the movement calculation node is launched to compute the required actions based on the input data, completing the control loop.

7.1 System Design

Figure 4 shows the system design for this project. The system integrates the RealSense RGB-D camera, ROS (and its tools), and the Franka Emika Panda robot. The RGB-D camera captures the environment and detects a specific chosen colour using a colour detection and tracking algorithm. This algorithm identifies and tracks the human wrist position in real-time. The detected position is then passed on to the "Panda Movement Calculation" node so it can be processed. The communication between these two nodes is done via a ROS topic called "wristPosition". This second module calculates the target position for the robot's end-effector, based on the tracked wrist position. This position is then sent to the Motion Planner, which will try to find a suitable trajectory for the robotic arm. The Planner will then transmit the appropriate joint angles and trajectory to the robotic arm. The state of the robot is also sent back to the "Panda Movement Calculation" to make adjustments based on the received state.

This modular design in ROS allows real-time communication between components. Each module communicates efficiently, ensuring smooth operation. The system's adaptability means it can be scaled for more complex

tasks. Figure 4 highlights the clear flow of data and processes, demonstrating the integration of hardware, algorithms, and simulation environments in this project.

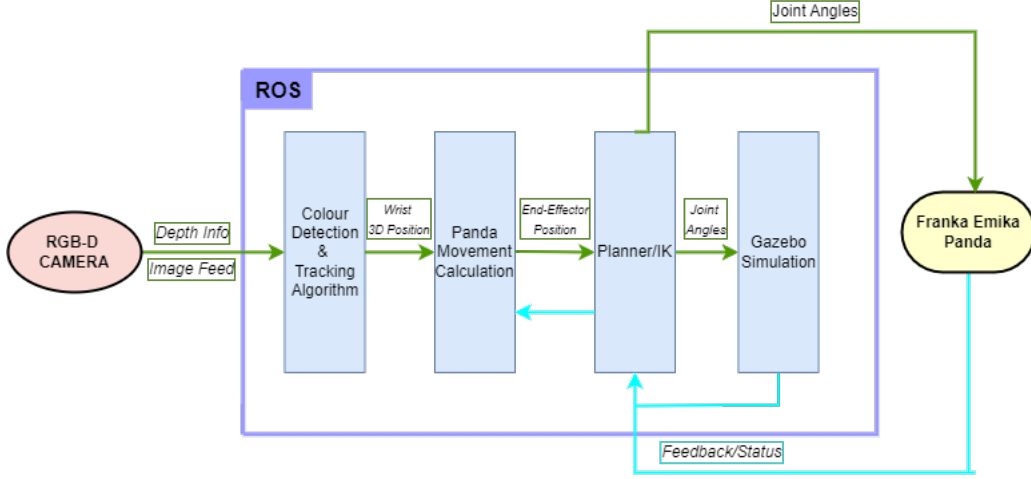


Figure 4: System Design

7.2 Wrist Tracking

This wrist tracking algorithm is designed to detect and track a coloured wristband placed on the wrist, using the RealSense RGB-D camera. The wristband, which is red, serves as the primary feature for detection. The algorithm’s core logic revolves around colour detection, using the HSV (Hue, Saturation, Value) colour space to isolate the red colour of the wristband.

First, the algorithm captures RGB and depth frames from the RealSense camera. The RGB frame is then converted to the HSV colour space, where colour segmentation becomes more robust. In HSV, the colour components are separated into distinct channels, making it easier to isolate specific colours. This is crucial because HSV allows better manipulation of colours under varying lighting conditions, which would be more difficult to achieve using the RGB colour model.

A crucial step in detecting the wristband is the creation of a colour mask using `'cv::inRange()'`. This function filters out all colours except those within a specified range, defined here as the typical hue for red. This process results in a binary image which assigns the value of '1' (white) to the pixels within the defined colour range, and '0' (black) to the pixels outside that range. This mask highlights areas of the image corresponding to the wristband, making it easy to locate and track. Figure 5 shows both the masked and RGB images. This method of masking is efficient and particularly effective for object tracking in a controlled environment.

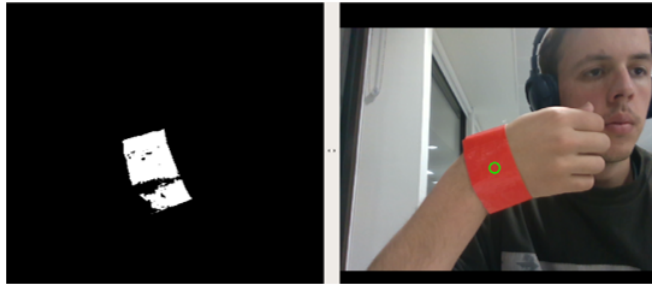


Figure 5: Tracking Visualisation on Rviz

After generating the mask, the algorithm uses OpenCV's `'findContours()'` function to detect the outline of the masked areas. The largest contour is assumed to be the wristband, and the algorithm calculates the centroid of this contour using image moments. The centroid is calculated by averaging all the pixel coordinates of the detected part. This point is then used to determine the wrist's position in the 2D image plane.

The depth information from the RealSense camera is used to estimate the distance to the wristband. Using the camera's intrinsic parameters, the 2D coordinates of the wristband's centroid are projected into 3D space. This provides the wrist's precise location in three-dimensional space relative to

the camera.

Finally, the 3D position is published to the 'objectPosition' topic, making it available to other nodes. As shown in Figure 5, the algorithm publishes both the original RGB image and the binary mask, which can be viewed on RViz. This allows for visual feedback for debugging or further analysis.

It is worth mentioning that OpenCV is a very useful tool in this algorithm by providing robust functions for image processing, contour detection, and colour conversion. Its flexibility and efficiency make it an ideal tool for computer vision tasks like object tracking. The use of HSV masking is key for accurate colour segmentation, making the algorithm resilient to changes in lighting conditions, which can be a major challenge in real-world applications.

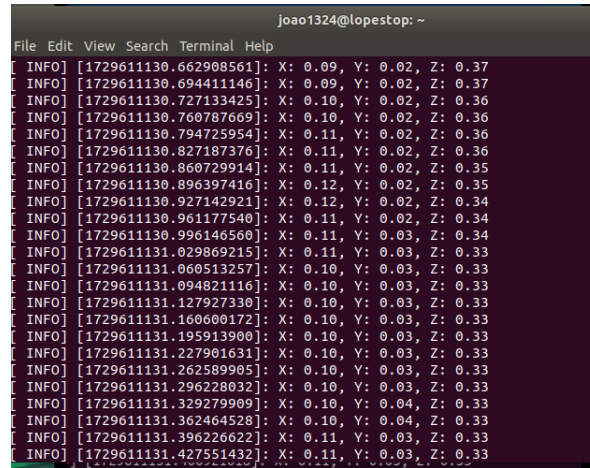
A terminal window titled 'joao1324@lopestop: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal displays a series of INFO messages, each containing a timestamp and 3D coordinates (X, Y, Z). The coordinates are: [1729611130.662908561]: X: 0.09, Y: 0.02, Z: 0.37; [1729611130.694411146]: X: 0.09, Y: 0.02, Z: 0.37; [1729611130.727133425]: X: 0.10, Y: 0.02, Z: 0.36; [1729611130.760787669]: X: 0.10, Y: 0.02, Z: 0.36; [1729611130.794725954]: X: 0.11, Y: 0.02, Z: 0.36; [1729611130.827187376]: X: 0.11, Y: 0.02, Z: 0.36; [1729611130.860729914]: X: 0.11, Y: 0.02, Z: 0.35; [1729611130.896397416]: X: 0.12, Y: 0.02, Z: 0.35; [1729611130.927142921]: X: 0.12, Y: 0.02, Z: 0.34; [1729611130.961177540]: X: 0.11, Y: 0.02, Z: 0.34; [1729611130.996146560]: X: 0.11, Y: 0.03, Z: 0.34; [1729611131.029869215]: X: 0.11, Y: 0.03, Z: 0.33; [1729611131.060513257]: X: 0.10, Y: 0.03, Z: 0.33; [1729611131.094821116]: X: 0.10, Y: 0.03, Z: 0.33; [1729611131.127927330]: X: 0.10, Y: 0.03, Z: 0.33; [1729611131.160600172]: X: 0.10, Y: 0.03, Z: 0.33; [1729611131.195913900]: X: 0.10, Y: 0.03, Z: 0.33; [1729611131.227901631]: X: 0.10, Y: 0.03, Z: 0.33; [1729611131.262589905]: X: 0.10, Y: 0.03, Z: 0.33; [1729611131.296228032]: X: 0.10, Y: 0.03, Z: 0.33; [1729611131.329279909]: X: 0.10, Y: 0.04, Z: 0.33; [1729611131.362464528]: X: 0.10, Y: 0.04, Z: 0.33; [1729611131.396226622]: X: 0.11, Y: 0.03, Z: 0.33; [1729611131.427551432]: X: 0.11, Y: 0.03, Z: 0.33.

Figure 6: Example of Tracked Points

7.3 Panda Movement Calculation

This implementation uses ROS and MoveIt to enable a Panda robotic arm to follow the motion of a wrist. A smoothing algorithm enhances the accuracy of wrist tracking as captured by the RealSense camera. At its core, the algorithm translates real-time position updates into precise end-effector movements, ensuring responsiveness while maintaining stability through filtering and constraints.

The algorithm begins by initialising ROS components and setting the Panda robot to a predefined "ready" position. This pose places the joints in a neutral and relaxed state. This means that the joints are not under significant strain and are positioned in a way that makes the robot ready to move more freely. This position serves as the reference point for subsequent transformations between the camera and robot workspaces.

When subscribing to the wrist's position topic ('objectPosition'), the programme calls back the method 'positionCallback'. This function uses the obtained positions as the real-time displacement from the 'ready' pose. In other words, the position received from the camera is added to the 'ready' pose, after processed. This approach simplifies the mapping of input positions into meaningful robot movements.

An important feature of the implementation is the application of Exponential Moving Average (EMA) smoothing to the incoming wrist position data. This technique filters out abrupt changes and sensor noise, ensuring smoother robot trajectories. The EMA formula ($\alpha * NewPoint + (1 - \alpha) * PreviousPoint$) blends the latest position data with the previous smoothed values, controlled by a smoothing factor (α). This balance between responsiveness and stability ensures the arm reacts promptly without overshooting or erratic behaviour. For this implementation, the value of 0.7 was assigned to α . This decision allows for more responsiveness while still reducing possible noise.

After smoothing, the algorithm constrains the calculated positions within

bounds that reflect the robot's workspace limits. The 'X' and 'Y' coordinates are clamped between -0.8 and 0.8, while 'Z' is restricted to values between 0 and 0.8. This means that if the target position coordinates are below the minimum allowable value, this minimum value will be used instead. And if it's above the maximum allowable limit, then this maximum value is kept. This avoids unachievable or unsafe targets, ensuring the robot operates within its physical constraints.

Depth adjustment is another critical aspect. This implies modifying the smoothed X-coordinate (which corresponds to the camera depth value) based on the initial registered depth. In this alteration, the programme saves the fifth valid depth, allowing time for the subject to set their wrist in the desired position. This recorded depth, called 'baseDepth', is then subtracted to the smoothed X-coordinate. This adjustment allows for 'X' to hold zero and negative values, since the depth obtained by the camera is always positive. This way the robotic arm can also move "backwards" and not just around the front.

The calculated target positions are then translated into a target pose for the Panda robot, including orientation and Cartesian coordinates. The MoveIt API facilitates planning and executing these movements, which is performed by the Motion Planner. By using asynchronous motion execution ('moveGroup.asyncMove()'), the system ensures non-blocking behaviour, allowing it to process subsequent commands while the robot moves. Otherwise the subscription to the topic would cease. There is also a position tolerance of 1cm. This facilitates the motion planning process while still achieving optimal results.

The system also incorporates a monitoring mechanism. The 'statusCallback' function tracks the robot's movement status by receiving messages published by the Motion Planner. This information is then used to prevent the programme to calculate a new position if the trajectory is being planned or executed, by setting a flag on. This flag is then used to condition

the 'positionCallback' method: if on it skips the method, if off it processes the position and moves the robot. This allows for MoveIt to only plan and execute when needed whilst also maintaining the position subscription.

Additionally, a logging procedure was also implemented. This mechanism uses the state messages from the 'statusCallback' function to record the number of successful and failed plans. Included in the 'positionCallback' method, another mechanism registers the camera points, the processed points, and the actual points the robot reaches. This data is then analysed for monitoring purposes.

Overall, this implementation demonstrates a robust integration of motion planning, filtering, and real-time control for robotic applications. The use of EMA smoothing ensures precise movements while minimizing noise, and the transformation between camera and robot workspaces simplifies alignment.

7.4 Motion Planner

Initially, the Open Motion Planning Library (OMPL) was used to plan robotic motion. While OMPL is efficient and versatile, its paths prioritize exploration and obstacle avoidance rather than smooth, natural trajectories. This often resulted in random, erratic movements unsuitable for applications requiring human-like motion. Though efficient in computation, OMPL's lack of emphasis on path smoothness made it a less ideal choice for scenarios where natural motion is critical.

To improve motion quality, the Pilz Industrial Motion Planner (PIMP) was adopted for its ability to generate predictable, smooth, and repeatable trajectories. Tailored for industrial robots, Pilz excels in integrating velocity, acceleration, and Cartesian constraints into its planning process. It ensures high efficiency, typically taking only a fraction of a second to a few seconds depending on task complexity. This low computational cost keeps the system responsive, even in dynamic settings, while enabling human-like precision and flow. Pilz's intuitive configuration within the MoveIt framework makes

it highly adaptable and reliable for collaborative workflows.

The LIN (linear) motion mode in Pilz further enhances motion quality. In this setting, the robot’s Tool Center Point (TCP) moves along straight-line trajectories, mimicking how humans naturally extend their arms during tasks. LIN motion avoids unnecessary rotational and oscillatory movements, ensuring smooth transitions between positions. While LIN paths might take slightly longer to plan compared to simpler configurations, the difference is negligible compared to the substantial improvement in motion quality. Parameters such as velocity, acceleration, and jerk limits can be fine-tuned, ensuring trajectories are both efficient and fluid.

In addition to the LIN setting, the Pilz Industrial Motion Planner (PIMP) provides two other modes: PTP (Point-To-Point) and CIRC (Circular Motion). PTP focuses on moving the robot from one position to another along the shortest path in joint space, prioritizing speed and efficiency. While it’s fast, it may lack the smoothness needed for human-like motion. CIRC generates circular trajectories, ideal for tasks like welding or painting, where consistent arcs are necessary. Though more computationally intensive, it does offers great fluidity. While these modes have their applications, the LIN setting remains the preferred choice for producing the most natural, human-like movements due to its ability to ensure predictable and smooth linear motion in Cartesian space.

By integrating Pilz, the system benefits from both computational efficiency and human-like motion. The reduced randomness and smooth trajectories improve task accuracy and realism, making the robot’s movement more natural. This balance between planning time and motion quality significantly enhances the system’s overall performance, enabling fluid human mimicking.

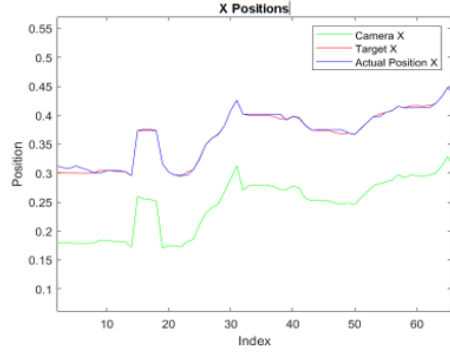
8 Results

8.1 Positional Analysis

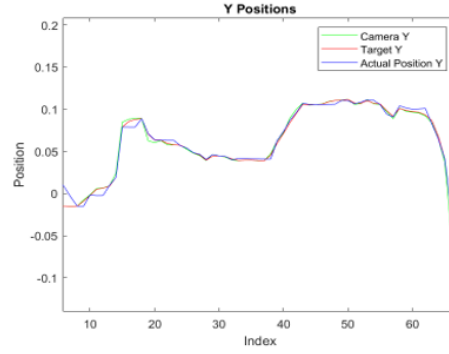
The results shown in Figure 7 highlight the relationships and changes between the camera's input, the processed target positions, and the robotic arm's actual movements. The green line represents the camera's raw position data, the red line corresponds to the processed and smoothed target positions sent to the robotic arm, and the blue line represents the robot's actual movement.

The difference between the green camera line and the other two lines reflects the offset introduced by the 'ready' position, which is used as a reference point for motion. Since in the 'ready' position the Y-coordinate is zero, there is no offset in the Y-axis graph. This results in an almost perfect alignment between the green and red lines in the Y-axis graph, with the only difference stemming from the smoothing and processing applied to the camera data.

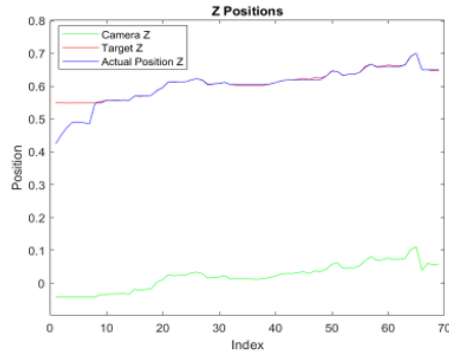
Across all graphs, the green camera line is slightly noisier, showing the real-time fluctuations inherent in camera-based tracking. Despite this, the red line demonstrates effective smoothing and remains highly responsive to changes in the green line, indicating that the processing does not introduce significant delays. The blue line, representing the actual robotic arm position, closely follows the red line with occasional minor deviations. These deviations are within a 1cm tolerance threshold, allowing for smooth and accurate operation without unnecessary oscillations.



(a) X



(b) Y



(c) Z

Figure 7: 3D points over time

Overall, these results confirm the system's ability to interpret and smooth camera data effectively while maintaining high responsiveness. The robotic arm demonstrates very accurate tracking of the processed target positions, achieving robust performance. The minimal gaps between the target position and actual movement lines underline the accuracy of the motion planning and control systems. Meanwhile, the camera data, though noisier, provides the necessary responsiveness for dynamic tracking. These results validate the system's reliability in aligning the robotic arm's motion with dynamic targets while maintaining smooth operation and accounting for small tolerances.

8.2 Motion Planner Success

The graph presented in figure 8 illustrates the performance of the Pilz industrial motion planner. The three-dimensional scatter plot shows the results of motion planning with axes labelled Target X, Target Y, and Target Z. Data points are colour-coded: green for successful plans and red for failed plans.

Out of these 200 planning attempt samples, 82% were successful. This positive result can also be deduced from graph, since the majority of the data points are green, indicating a high success rate. Either way, it is clear that the motion planner performed well overall.

However, there are noticeable clusters of red points, representing failed plans, particularly around the edges of the graph. This pattern indicates that the motion planner struggles with boundary conditions or extreme positions. This could be due to limitations in the planner's algorithm or physical constraints of the robotic system.

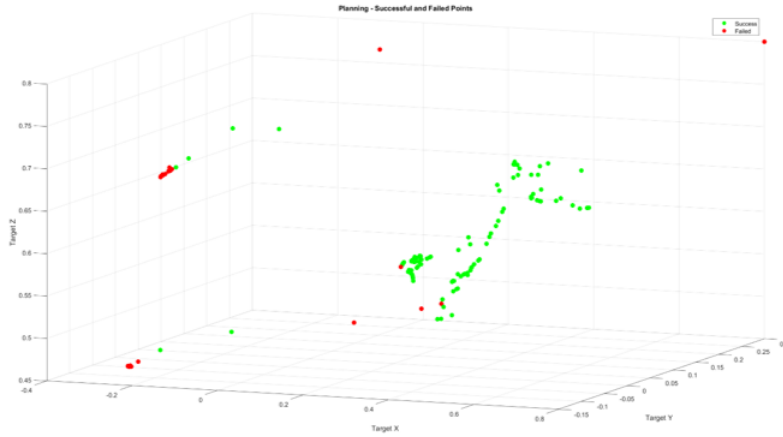


Figure 8: Successful vs Failed Positions

The samples used to generate this graph were obtained when the tracked wrist moved slowly. The slow movement contributed to the high success rate. It allowed the planner to make precise adjustments and avoid errors. If the wrist had moved faster, joint limits might have been exceeded, resulting in more frequent errors and a higher number of failed plans. This consideration is important for practical applications. It highlights the need to balance speed and accuracy in motion planning.

Overall, the graph provides a clear visual representation of the motion planner’s performance. It highlights both its strengths and areas for improvement. The concentration of failed plans around the edges suggests that further refinement of the planner’s algorithms or adjustments to the system’s constraints may be necessary to achieve more consistent performance across the entire range of motion. This analysis underscores the importance of considering both the speed of movement and the spatial constraints in optimizing robotic motion planning.

9 Challenges

The challenges encountered in this project covered technical, hardware, and conceptual difficulties, each requiring a tailored solution.

One of the first challenges involved configuring the Gazebo simulation environment. Gazebo relies on accurate URDF (Unified Robot Description Format) files to simulate robots and their environments. These files describe the robot’s structure, joints, sensors, and visual properties. In this case, errors in the URDF file led to issues during the setup. Debugging these errors was time-consuming, particularly since this was the first experience working with ROS. The problem was eventually resolved using the MoveIt Setup Assistant GUI, a tool designed to help users configure robots for motion planning. This assistant simplifies URDF validation and configuration by providing an intuitive interface to adjust parameters and generate necessary files for MoveIt integration. This approach not only solved the immediate issue but also made the robot ready for planning and execution tasks.

Another significant challenge involved Hand-to-Eye Calibration, an essential step for precise robotic manipulation in workspace environments. The MoveIt Hand-to-Eye Calibration package is designed to determine the transformation between the camera’s coordinate system and the robot’s base frame. This allows the robot to accurately interpret object locations captured by the camera and convert them into the robot workspace. Typically, this involves tracking a calibration object, such as an ArUco marker, and using this data to compute the transformation matrix. Although the ArUco target tracking worked successfully, MoveIt failed to integrate this data into the calibration process, likely due to compatibility or configuration issues. As an alternative, a simplified transformation approach was developed and implemented. This approach allowed the project to proceed successfully and is described in Section 7 of this report.

The depth camera malfunction presented another hurdle. Although it operated well at distances under 0.5 meters, beyond this range, it returned

highly inaccurate and unpredictable depth values. An implementation of a correction algorithm was attempted. This consisted on comparing the new depth value to the previous ones and, if needed, applying the correction based on that difference. This failed because the overshooting values lacked a consistent pattern. This forced a limitation on testing, ensuring all movements remained within the camera's functional range.

These challenges demonstrated the complexity of integrating diverse technologies in robotics. They highlighted the need to understand underlying software tools like Gazebo and MoveIt while also adapting to hardware limitations. Each issue offered valuable lessons in troubleshooting, improvisation, and the importance of having alternative strategies to achieve project goals.

10 Future Work

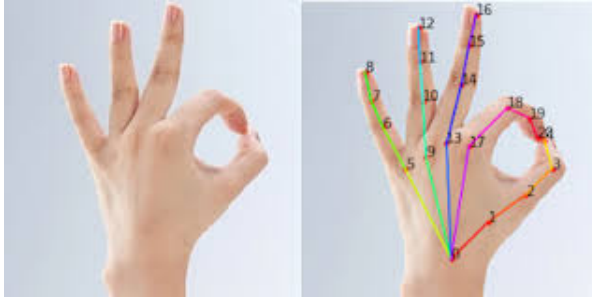
While the simulation in Gazebo provided a strong foundation for testing, implementing this project on a physical robot would be more effective to evaluate how well the system performs in the real world. Hardware factors like sensor noise, communication delays, and physical inaccuracies would provide a more realistic assessment. Additionally, a real-world implementation could help fine-tune the transformation and movement precision observed in the simulations.

Another promising direction for future work is improving the wrist tracking system by incorporating an AI-based model. Using AI could enhance tracking accuracy and extend the system’s capabilities to include hand orientation detection. This would allow for more human-like movements, improving the fluidity and natural appearance of the robot’s motions.

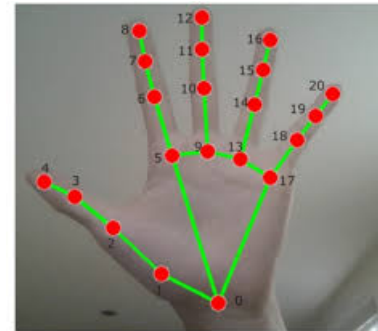
There was an initial attempt to use OpenPose and MediaPipe for hand tracking, but compatibility issues arose due to the use of ROS Melodic, an older version of the framework. OpenPose is a library that uses deep learning to estimate human body and hand keypoints. It processes input frames, identifies key joints, and generates precise skeleton models (shown in Figure 9a), including hand positions and orientations.

MediaPipe offers a lightweight, real-time framework for hand tracking. It uses machine learning to identify and map 21 hand landmarks (Figure 9b), even in complex or dynamic environments. MediaPipe is more efficient and runs on devices with limited computational resources, making it a suitable option for real-time applications.

Addressing these compatibility issues or upgrading to a newer ROS version could resolve these challenges. Integrating AI-based hand tracking would significantly enhance the system’s adaptability and performance, paving the way for future improvements in robot movement and interaction.



(a) OpenPose Skeleton



(b) MediaPipe Landmarks

Figure 9: AI Models

11 Conclusion

This project demonstrated a simplified approach to robotic wrist trajectory tracking. By leveraging real-time color-based tracking and straightforward motion planning, the Franka Emika Panda robotic arm was able to follow human wrist movements with high accuracy. The system achieved an 82% success rate in motion planning and showcased smooth trajectories under controlled conditions. The use of ROS, MoveIt, and Gazebo provided a robust framework for integration and testing. However, several challenges arose during development. Depth sensor inaccuracies limited the robot’s operational range, while calibration issues complicated the transformation between camera and robot workspaces. These obstacles highlighted the complexity of integrating hardware and software in robotic systems. Future work should focus on improving wrist tracking accuracy by incorporating AI-based models, such as OpenPose or MediaPipe. This would enable the system to track hand orientation and perform more complex tasks. Real-world implementation is also recommended to evaluate the system under realistic conditions, addressing factors like sensor noise and physical inaccuracies. Overall, this project achieved its primary objective of real-time robotic motion tracking. It provides a foundation for future research in human-robot interaction and opens the door to incorporating advanced techniques like Learning from Demonstration in robotic control systems.

References

- [1] I. RealSense, “Beginner’s guide to depth (Updated) — intelrealsense.com,” <https://www.intelrealsense.com/beginners-guide-to-depth/>.
- [2] “Franka Emika Panda robot - RoboDK — robodk.com,” <https://robodk.com/robot/Franka/Emika-Panda>.
- [3] “Franka Robots- POMO Robotics — pomorobotics.com,” <https://www.pomorobotics.com/robots/frankpanda/>.
- [4] “ROS/Introduction - ROS Wiki — wiki.ros.org,” <https://wiki.ros.org/ROS/Introduction>.
- [5] “MoveIt Open Source Robot Motion Planning Robot Sim — picknik.ai,” <https://picknik.ai/moveit>.
- [6] “About Gazebo,” <https://gazebo.org/about>.
- [7] “ROS Gazebo: Everything You Need To Know - Robotic Simulation Services — roboticsimulationservices.com,” <https://roboticsimulationservices.com/ros-gazebo-everything-you-need-to-know/>.
- [8] M. Quigley, B. Gerkey, and W. D. Smart, *Programming Robots with ROS*. O’Reilly Media, 2015.